SportsPulse

Cricket Performance Analytics using Data Structures & Algorithms

A simple Java platform that uses cricket data to demonstrate searching, pattern matching, optimization, graph algorithms and randomized techniques.

DSA Semester Project

Problem & Motivation

Why use DSA for sports data?

- Cricket datasets contain a large number of match, batting and bowling records.
- Finding useful information manually becomes difficult as the data grows.
- Different problems need different algorithms for searching, matching, optimization and ranking.
- SportsPulse uses cricket as a familiar domain to demonstrate these DSA solutions.

Project Overview

What SportsPulse does

Input

Cricket CSV data plus user-selected names, keywords, budgets and numeric values.

Processing

Six DSA modules apply different algorithmic techniques to the data.

Output

Search results, repeated patterns, corrected names, optimized selections, assignments, rankings and samples.

Main Goal

Show the practical use of advanced DSA concepts through one sports domain.

Data Used

Dataset and application input

match.csv

Match-level information such as match, date, teams, runs and overs.

bat.csv

Batting records including player, runs, balls, fours, sixes and dismissal.

ball.csv

Bowling records including bowler, runs, wickets, overs and economy.

Dataset size

6,197 match records • 52,492 batting records • 36,149 bowling records.

Modules 1 & 2 — String Analysis

Finding players, events and repeated text

String Searching

KMP, Rabin–Karp and Aho–Corasick search player names and cricket keywords.

Suffix Structures

Suffix Array and Kasai LCP identify repeated patterns in cricket-related text.

Why it matters

Useful for fast text search, commentary search and finding recurring phrases.

Output

Keyword occurrences and the longest repeated phrase found in the selected text.

Module 3 — Dynamic Programming

Three different optimization problems

Levenshtein Distance

Finds a close player name when the input contains a spelling mistake, e.g. Tendulker → Tendulkar.

Bitmask DP

Demonstrates budget-based squad selection using a derived cost/rating model.

Matrix Chain Multiplication

Finds an efficient order for multiplying a sequence of matrices.

Output

Corrected player and stats, selected players, and minimum multiplication cost.

Module 4 — Network Flow & Matching

Capacity and assignment problems

Edmonds–Karp

Uses a simulated stadium-entry network to calculate maximum possible flow.

Bipartite Matching

Matches available players with tactical roles without reusing the same role.

Input

User selection and player information derived from the cricket dataset.

Output

Maximum flow value and player-to-role assignments.

Modules 5 & 6 — Graph & Randomized Algorithms

Conflicts, ranking and large streams

NP Approximation

Uses Vertex Cover 2-Approximation and Independent Set on a simulated player-conflict graph.

Randomized QuickSort

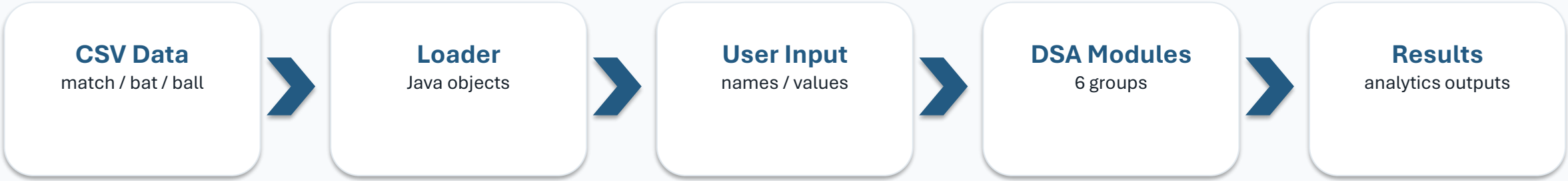
Sorts a custom cricket performance score to create a leaderboard.

Prefix Scan

Calculates cumulative runs from a sequence of scoring values.

Complete System Flow

From cricket data to DSA results



The project combines real cricket records with simulated inputs where needed to demonstrate specific DSA problems.